

基础工具集与常用数据集

基础工具集与常用数据

基础工具集

- NLTK
- LTP
- PyTorch

常用数据集

- Wikipedia
- Common Crawl

□ NLTK(Natural Language Toolkit):

- 对英文文本数据进行处理常用工具

- 是一个 Python 模块，提供了

- ① 多种语料库(Corpora)和词典(Lexicon)资源

- 生文本、PennTreebank样例

- WordNet

- ② 基本的自然语言处理工具集，包括：

- 分句

- 标记解析(Tokenization)

- 词干提取(Stemming)

- 词性标注(POS Tagging)

- 句法分析(Syntactic Parsing)

- ...

□ Natural Language Toolkit

□ <https://www.nltk.org/>



<https://data-flair.training/blogs/nltk-python-tutorial/>

□ NLTK安装 `$ pip install nltk`

1. 语料库和词典资源下载

```
>>> import nltk  
>>> nltk.download()
```

① 停用词(Stop words)：对表达语言的含义不重要的词

□ 例子：冠词 “a”、“the”，介词 “of”、“to” 等

□ 处理方法：对语言进行更深入的处理之前，删除

□ 加快处理的速度，减小模型的规模

□ 引入停用词表： `from nltk.corpus import stopwords`

□ 查看停用词表： `stopwords.words("english")`

② 常用语料库：NLTK提供多种语料库(文本数据集)，如图书、电影评论和聊天记录等,可被分为两类：

□未标注数据库,两种访问方式：

- ① 直接访问语料库的原始文本文件
- ② 调用NLTK提供的相应功能，如获得Gutenberg语料库中简·奥斯汀所著的小说Emma原文

□ `from nltk.corpus import gutenberg`

□ `gutenberg.raw("austen-emma.txt")`

□人工标注数据库

□人工标注的关于某项任务的结果

□如句子极性语料库(sentence_polarity)

□包含了10,662条来自电影领域的用户评论句子以及相应的极性信息（褒义或贬义）

□ `sentence_polarity`:基本的数据访问方法

□ `sentence_polarity.categories`、`sentence_polarity.words`、`sentence_polarity.sents`

③ 常用词典

□ WordNet

- 构建单位：普林斯顿大学

- 主要特色：

- ① 定义了同义词集合(Synset)
- ② 为每一个同义词集合提供了简短的释义
- ③ 不同同义词集合之间还具有一定的语义关系

□ SentiWordNet

- 为WordNet中每个同义词集合人工标注了三个情感值(褒义、贬义和中性)

2. 常用的自然语言处理工具集

- ① 分句
- ② 标记解析
- ③ 词性标注
- ④ 命名实体识别
- ⑤ 组块分析
- ⑥ 句法分析

□ 更多英文自然语言处理工具集

□ CoreNLP、spaCy等

语言技术平台 (Language Technology Platform , LTP)

<http://ltp.ai>

高效、高精度中文自然语言处理基础平台

中文词法、句法、语义分析等6项自然语言处理核心技术



2020年7月发布4.0版本

基于预训练模型

多任务学习机制

安装

\$ pip install ltp

在线演示
直观了解语言技术平台能为你做什么

国务院总理李克强调研上海外高桥时提出，支持上海积极探索新机制。

句子视图 篇章视图 XML视图

☒ 词性标注 ☒ 命名实体 ☒ 句法分析 ☒ 语义角色标注 ☒ 语义依存分析

应用1句子1: 国务院总理李克强调研上海外高桥时提出，支持上海积极探索新机制。

标签释义

Tag	关系类型	Description
Ag	施事关系	Agent
Dir	趋向角色	Direction
ePurp	目的关系	event Purpose
Feat	描写角色	Description
Mann	方式角色	Manner
mPunc	标点标记	Punctuation Marker
mTime	时间标记	Time
Nmod	名字修饰角色	Name-modifier
Prod	成事关系	Product
Root	根节点	Root

LTP调用示例

□中文分词

□分句

□词性标注

```
>>> sentences = ltp.sent_split(["南京市长江大桥。", "汤姆生病了。他去了医院。"])  
# 分句  
>>> print(sentences)  
['南京市长江大桥。', '汤姆生病了。', '他去了医院。']  
>>> segment, hidden = ltp.seg(sentences)  
>>> print(segment)  
[['南京市', '长江', '大桥', '。'], ['汤姆', '生病', '了', '。'], ['他', '去', '了', '医院', '。']]  
>>> pos_tags = ltp.pos(hidden) # 词性标注  
>>> print(pos_tags) # 词性标注的结果为每个词所对应的词性，LTP使用的词性标记集与  
# NLTK不尽相同，但基本大同小异  
[['ns', 'ns', 'n', 'wp'], ['nh', 'v', 'u', 'wp'], ['r', 'v', 'u', 'n', 'wp']]
```

□ Facebook开发的开源深度学习库

□ <https://pytorch.org/>

□ 基于张量 (Tensor) 的数学运算工具包

□ GPU加速的张量计算

□ 自动进行微分计算

□ PyTorch的优势

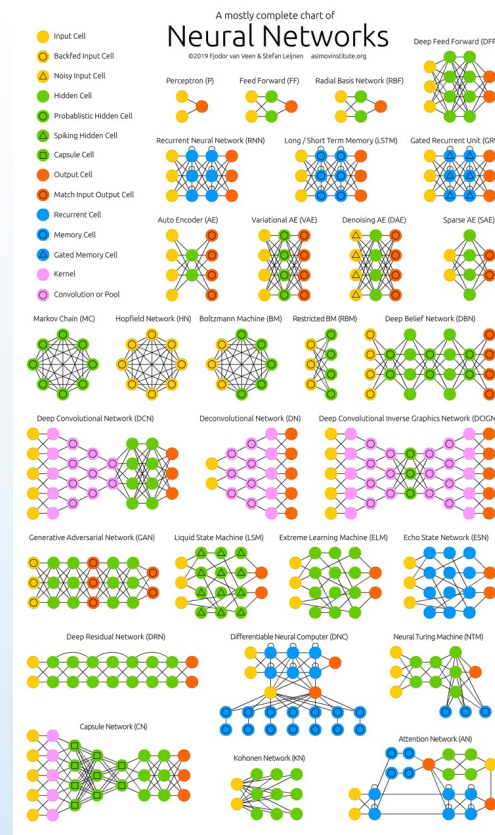
□ 框架简洁

□ 入门简单，容易上手

□ 支持动态神经网络构建

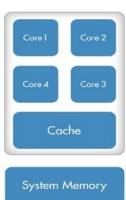
□ 与Python语言无缝结合

□ Debug方便

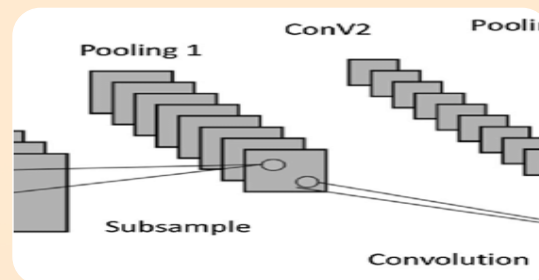
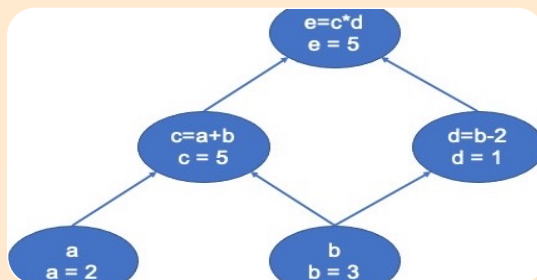
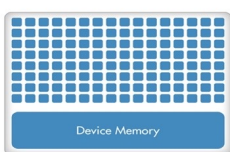


PyTorch的三要素

CPU (Multiple Cores)



GPU (Hundreds of Cores)



张量 (Tensor)

- 数据存储
- 支持GPU

表达式 (Expression)

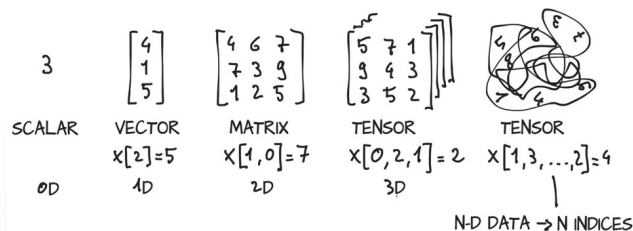
- 数据运算
- 自动微分计算

模块 (Module)

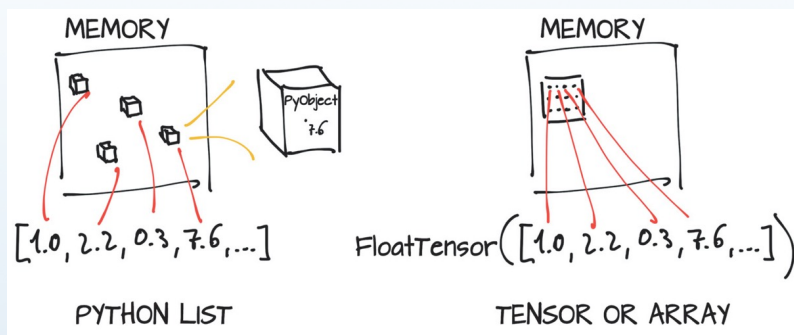
- 神经网络层
- 支持自定义

张量 (Tensor)

□ 又称为多维数组



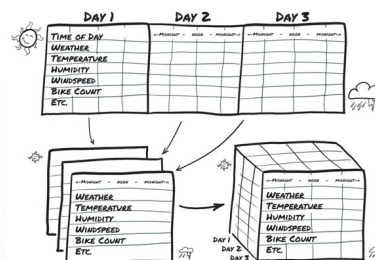
□ 与Python列表的区别



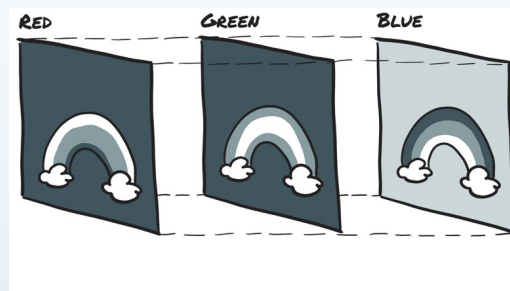
□ 存储方式完全不同

□ 可表示现实世界中的各种数据

□ 表格、时间序列



□ (彩色) 图像



batch = torch.zeros(100, 3, 256, 256, dtype=torch.uint8)

- ① 张量的创建
- ② 张量的基本运算
 - ▣ 按照对应的元素进行
- ③ 自动微分
- ④ 调整张量形状
- ⑤ 广播机制
- ⑥ 索引与切片
- ⑦ 升维与降维

PyTorch作为张量库

□支持各种张量操作(类似NumPy)

- 创建张量

- 索引、切片

□支持GPU

- 快！

$$M * M * M$$

$$M \in \mathbb{R}^{1000 \times 1000}$$

Numpy

```
In [2]: M = numpy.random.randn(1000,1000)

In [3]: timeit -n 500 M.dot(M).dot(M)
500 loops, best of 3: 30.7 ms per loop
```

PyTorch

```
In [4]: N = torch.randn(1000,1000).cuda()

In [5]: timeit -n 500 N.mm(N).mm(N)
500 loops, best of 3: 474 μs per loop
```


表达式 (Expression)

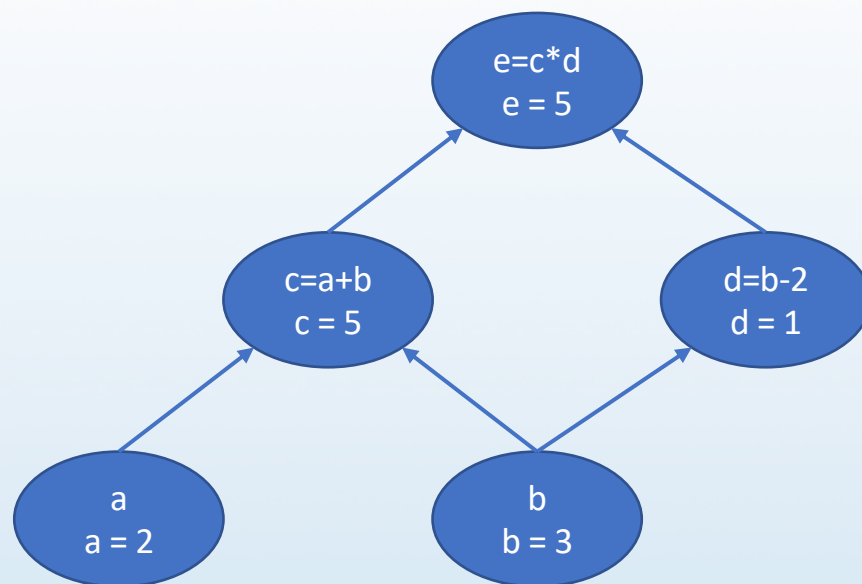
□ 通过 **计算图** 描述表达式

□ 如 : $e = (a + b) * (b - 2)$

□ 计算图中每个节点执行一个运算

□ 前向运算 (Forward) : 根据输入获得输出

```
>>> a = torch.tensor([2.])
>>> b = torch.tensor([3.])
>>> c = a + b
>>> d = b - 2
>>> e = c * d
>>> print(c, d, e)
tensor([5.], grad_fn=<AddBackward0>)
tensor([1.], grad_fn=<SubBackward0>)
tensor([5.], grad_fn=<MulBackward0>)
```



表达式 (Expression)

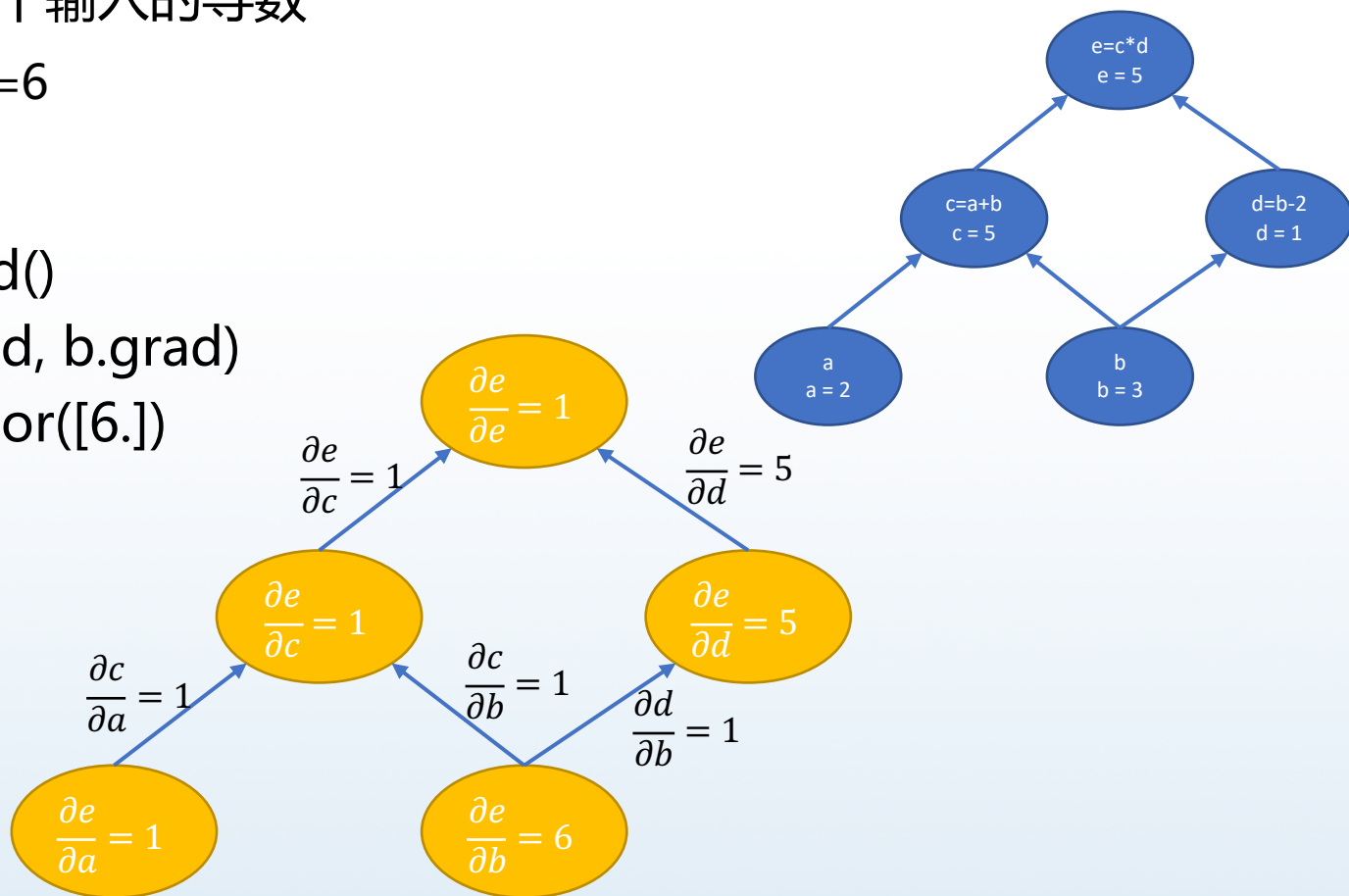
反向传播 (Back-propagation)

计算输出对每个输入的导数

$$\frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial b} = 6$$

自动微分计算

```
>>> e.backward()  
>>> print(a.grad, b.grad)  
tensor([1.]) tensor([6.])
```



模块 (Module)

包名	功能	描述
torch	Tesnor / Expression	类似NumPy的张量库，支持GPU以及自动微分
torch.nn	Module	灵活的神经网络库，提供多种神经网络层
torch.optim	Module	多种优化算法
torch.utils	Module	数据集、数据加载等辅助工具

大规模预训练数据集

❑ 维基百科(Wikipedia):不同语言写成的网络百科全书

- ❑ 创建人: 吉米·威尔士与拉里·桑格两人合作

- ❑ 创建时间:

 - ❑ 2001年1月13日在互联网上推出网站服务

 - ❑ 2001年1月15日正式展开网络百科全书的项目

- ❑ 特点: 内容由人工编辑, 适合作为预训练的原始数据

❑ 原始数据的获取

- ❑ 快照(2020年10月23日)<https://dumps.wikimedia.org/>

文件名	内容	大小/MB
zhwiki-latest-abstract.xml.gz	所有词条摘要	≈ 147
zhwiki-latest-all-titles.gz	所有词条标题	≈ 33
zhwiki-latest-page.sql.gz	所有词条标题及摘要	≈ 204
zhwiki-latest-pagelinks.sql.gz	所有词条外链	≈ 890
zhwiki-latest-pages-articles.xml.bz2	所有词条正文	≈ 1,952

- ❑ 预训练语言模型主要使用的是维基百科的正文内容

大规模预训练数据集

□ 维基百科(Wikipedia)-语料处理方法

① 纯文本语料抽取-处理维基百科快照

- 目的：去除其中的图片、表格、引用和列表等非常规文本信息，最终得到纯文本的语料
- WikiExtractor：Python的工具包
 - 安装
 - 使用

② 中文繁简转换

- 中文维基百科中同时包含了简体中文和繁体中文的数据
- 如果使用者只需要获得简体中文数据，需要将纯文本语料中的繁体中文内容转换为简体中文
- 转换工具：OpenCC

大规模预训练数据集

□ 维基百科(Wikipedia)-语料处理方法

③ 数据清洗

□ 对象：纯文本中包含一些乱码或机器字符

□ 处理的错误类型：

□ 删除空的成对符号

□ 例如 “ () ” “ 《 》 ” “ 【 】 ” “ [] ” 等

□ 删除 < br > 等残留的HTML标签

□ 注：不删除以 “ < doc id > ” 和 “ < /doc > ” 为开始的行，因其表示文档的开始和结束，能为某些预训练语言模型的数据处理提供至关重要的信息

□ 删除不可见控制字符

□ 免意外导致数据处理中断

大规模预训练数据集

- Common Crawl数据
(<https://commoncrawl.org/>)
 - 大规模网络爬虫数据集
 - 使用Facebook的CC-Net工具下载和处理
 - https://github.com/facebookresearch/cc_net

更多的数据集

- NLTK：模型演示和简单的系统测试
- HuggingFace Datasets：更大规模的语料库集合
 - 数据集数目多：共收录了近200种语言的700多个数据集(2021.03)
 - 涵盖文本分类、机器翻译和阅读理解等众多自然语言处理任务
 - 兼容性好：
 - 可以直接被PyTorch、TensorFlow等深度学习框架调用
 - 可被pandas、NumPy等数据处理工具调用
 - 支持CSV、JSON等数据格式的读取，并提供了丰富、灵活的调用接口和数据处理接口
 - 数据读取效率高：可以在仅占用少量内存的条件下，高速地读取大量的数据。
 - 丰富的评价方法
 - 多种通用的评价方法，还针对不同的数据集提供了更有针对性的评价方法
 - 用户还可以增加自定义的评价方法



Thanks!
Q&A ?

wuxianyan@njau.edu.cn